

Polynomial Reductions

Raina Burton

University of Colorado Denver

Karp Reductions

Let X and Y be two decision problems with input instance I_X and output instance I_Y , respectively. A **Karp Reduction** is a polynomial-time computable function $\mathcal{R}_p : X \rightarrow Y$ where

I_X is a YES-instance of $X \iff I_Y = \mathcal{R}(I_X)$ is a YES-instance of Y . [1]

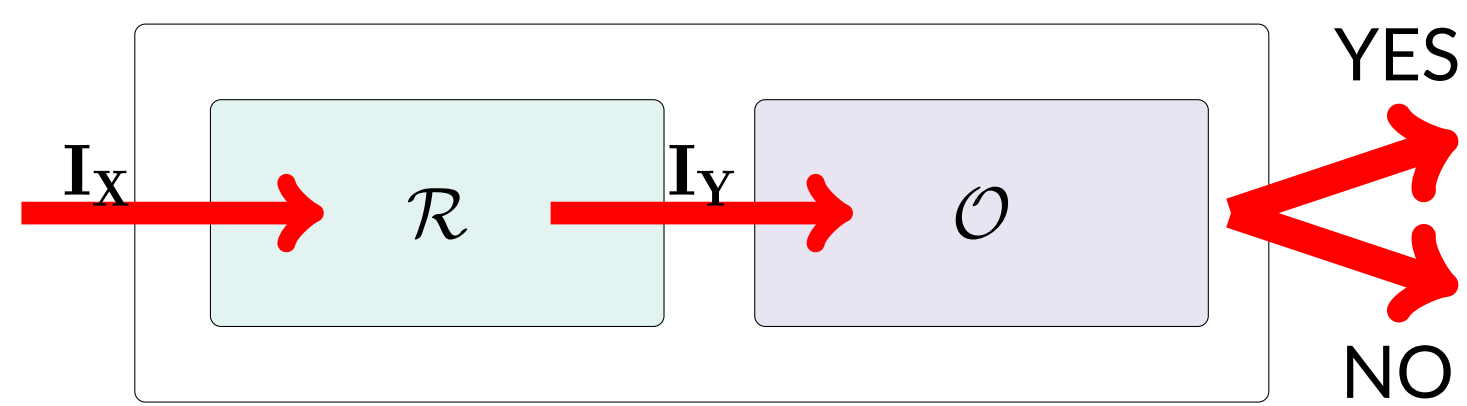
If there exists a Karp Reduction from X to Y , we say that X reduces to Y and use the notation:

$$X \leq_p Y$$

If $X \leq_p Y$, then Y is at least as hard as X .

Note that Karp reductions are transitive:

$$X \leq_k Y \text{ and } Y \leq_p Z \implies X \leq_p Z$$



NP-Completeness [2]

- P:** A problem X is in P if there exists an algorithm that solves it in polynomial time on a deterministic Turing machine. Note that

$$X \leq_p Y \text{ and } Y \in P \implies X \in P.$$

- NP:** A problem X is in NP if there exists a polynomial-time algorithm that solves it on a non-deterministic Turing machine.
- NP-Complete** A problem Y is NP-complete if
 - $Y \in NP$
 - $\forall X \in NP, X \leq_p Y$ [2]

Cook-Levin Theorem

The satisfiability problem is NP-complete.

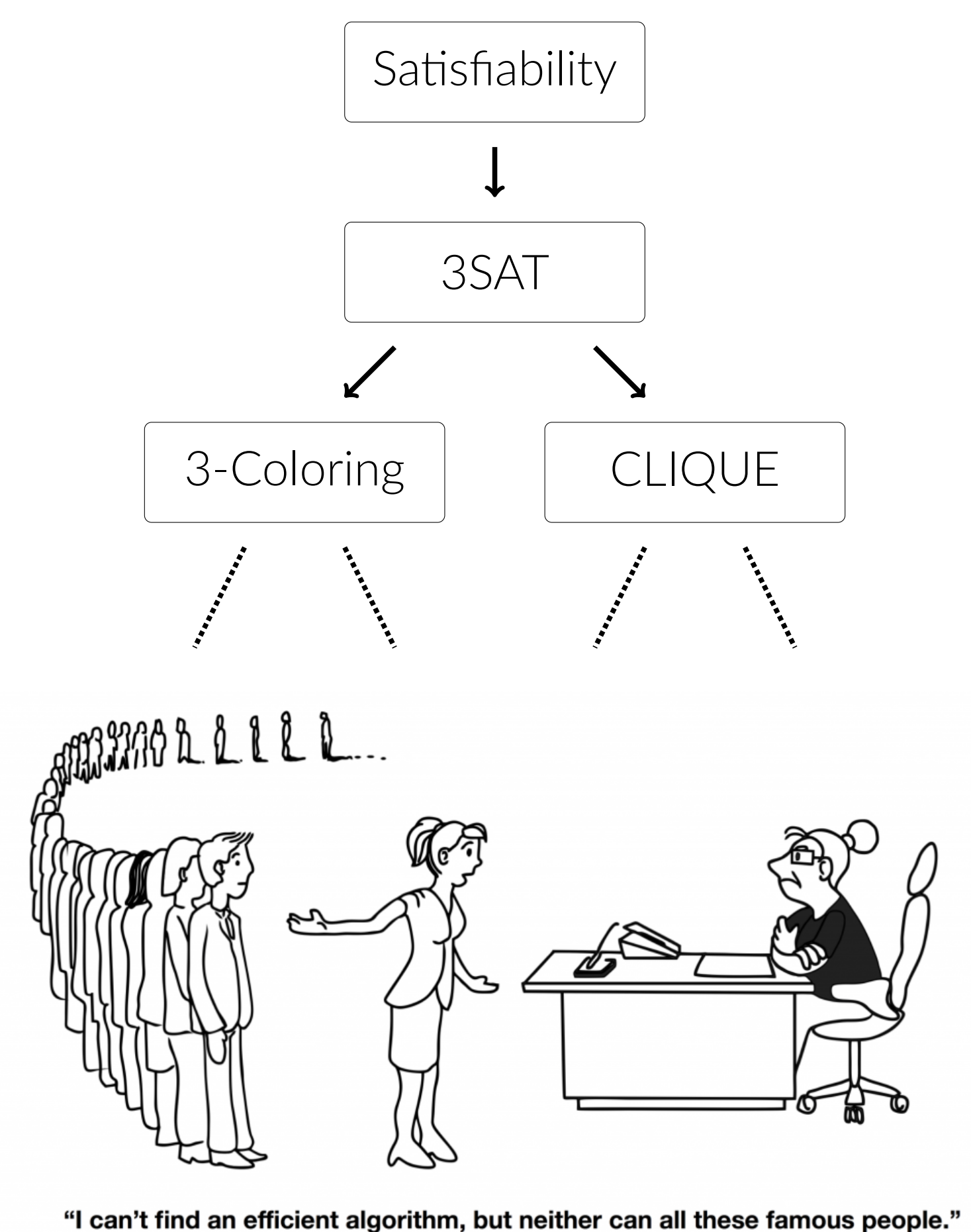


Figure 1. The "philosophy" of NP-Completeness. Cartoon by Stefan Szeider [4]

3SAT $\leq_k$ CLIQUE $\leq_k$ IND-SET Example

3SAT:

Def A literal is either a variable or a negation of a variable. A clause is a disjunction of literals

Given: A set $\mathcal{C}$ of c clauses, each with 3 literals

Determine: whether $\mathcal{C}$ is satisfiable

Consider the following instance, I_{3SAT} with $k = 3$ clauses.

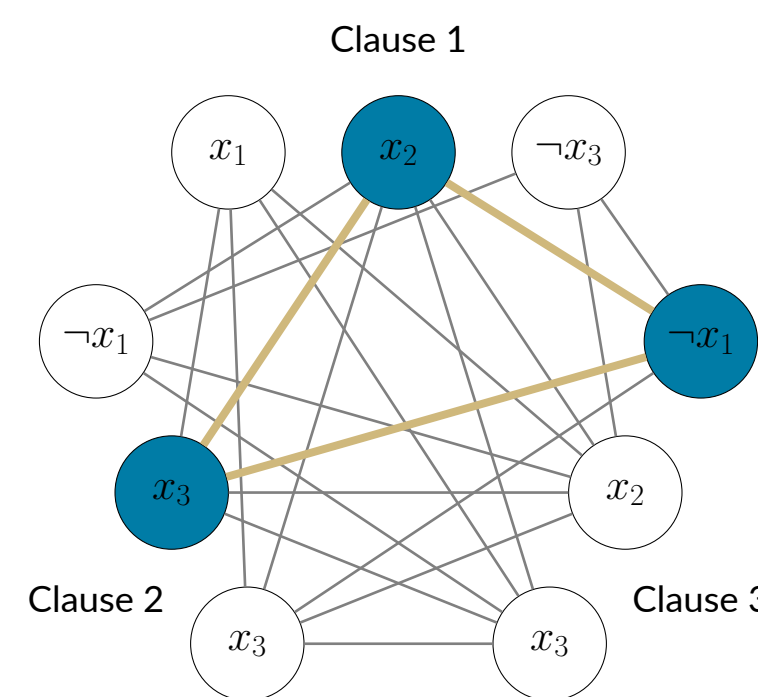
$$(x_1 \cup x_2 \cup \neg x_3) \cap (\neg x_1 \cup x_2 \cup x_3) \cap (\neg x_1 \cup x_2 \cup x_3)$$

The following algorithm performs a Karp reduction from any I_{3SAT} to a corresponding $I_{CLIQUE}(V, E)$:

```
procedure 3SAT  $\leq$  CLIQUE
  for Each clause  $c_i$  in  $I_{3SAT}$  do
    Create a node  $v_{ij}$  for  $j = 1, \dots, 3$ 
  end for
  for Each clause  $c_i$  in  $I_{3SAT}$  do
    for Each node  $v_{ij}$  in clause  $c_i$  do
      for Each node  $v_{km}$  not in clause  $c_i$  do
        if  $v_{ij}$  and  $v_{km}$  are consistent then
          Draw arc  $(v_{ij}, v_{k,m})$ 
```

The creation of nodes is linear in the number of literals, ie, $O(n)$. The drawing of edges is polynomial in the number of literals, specifically, $O(3k^2) = O(k^2)$.

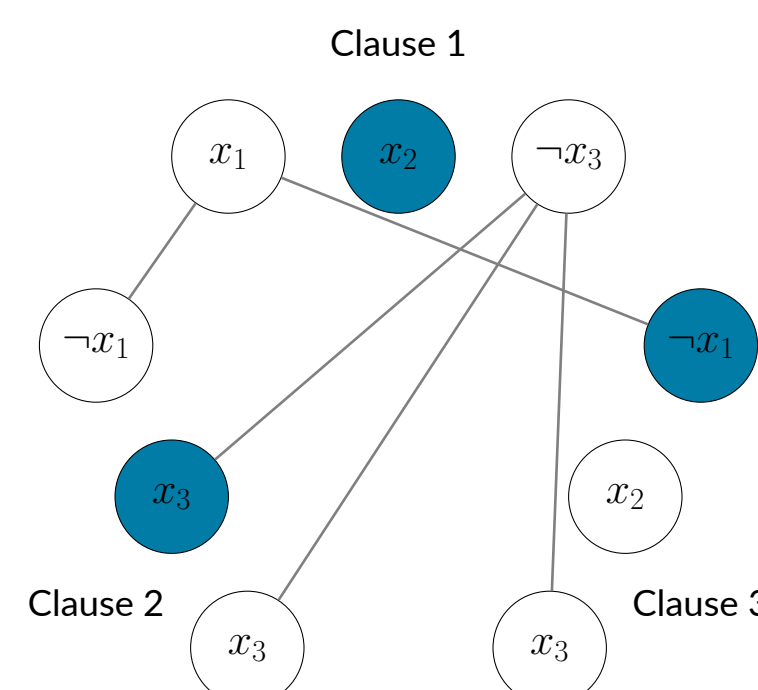
I_{CLIQUE} :



The following algorithm performs a Karp reduction from $I_{CLIQUE}(V, E)$ to $I_{IND-SET}(V, E)$:

```
procedure CLIQUE  $\leq$  IND-SET
  for Each node  $v_{ij}$  in  $I_{CLIQUE}(V, E)$  do
    for Each node  $v_{km}$  such that  $i \neq k$  and  $m \in \{1, 2, 3\}$ 
      in  $I_{CLIQUE}(V, E)$  do
        if  $(v_{ij}, v_{km}) \in E(CLIQUE)$  then
          Remove that edge
        else Add that edge
```

$I_{IND-SET}$:



This algorithm iterates through each pair of vertices. For each of the $\binom{n}{2}$ possible pairs, we check whether an edge currently exists, if not we create one. Thus this algorithm is in $O(1 \times \binom{n}{2}) = O(n^2)$

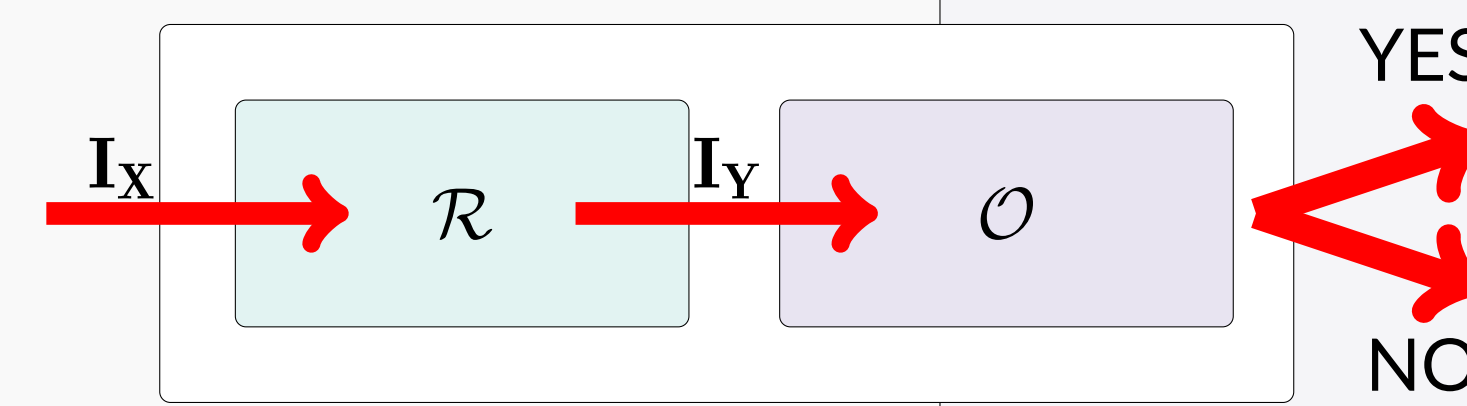
MAX-3SAT

Task: Given a set of clauses, each with 3 literals, find an assignment that satisfies the largest number of clauses.

The following algorithm demonstrates how self-reduction and Karp reduction can be combined to form a Cook reduction algorithm that solves the MAX-3SAT optimization problem given a decision oracle.

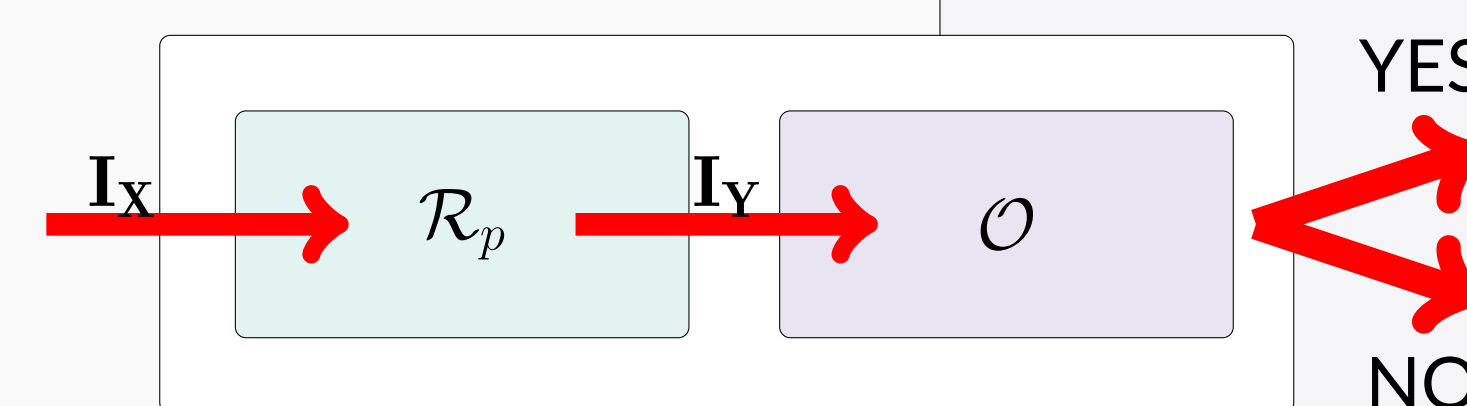
Cook Reduction: MAX-3SAT

```
1: procedure BINARY SEARCH FOR MAX K
2:   low  $\leftarrow$  1
3:   high  $\leftarrow$  total number of clauses
4:   while low  $\leq$  high do
5:     k  $\leftarrow$   $\lfloor (low + high) / 2 \rfloor$ 
6:     Ask the Oracle: "Is there a satisfying assignment with at least k clauses"
```



```
7:   if The oracle says "Yes",
8:     then low  $\leftarrow$  mid
9:   else
10:    high  $\leftarrow$  mid
```

```
1: procedure FIND WHICH CLAUSES FORM
   A SATISFIABLE SET OF SIZE MAX K
2:   Satisfiable set  $S = \{\}$ 
3:   while high  $\geq$  0 do
4:     for Each clause c do
5:       Ask the Oracle: "Does there exist a satisfiable set S such that  $\{c\} \cup S \in S$ ?"
```



```
6:   if The oracle says "Yes",
7:     then add c to set S
8:     high  $\leftarrow$  high - 1
```

Complexity Analysis: The algorithm is made of two main routines.

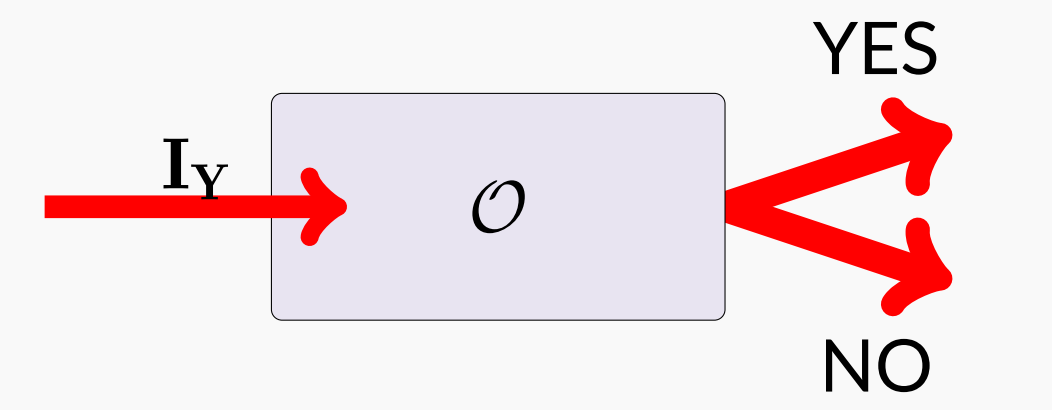
- Binary Search: $O(\log_2(n))$ queries to the oracle. Each query requires a reduction from K-3SAT to clique which takes $O(n^2)$ time, so this routine takes $O(n^2 \log_2(n))$

The the binary search routine runs in $O(\log_2(n))$ and the routine

Self Reductions

Given a search problem X and and a decision oracle $\mathcal{O}_X$ for X , a self reduction is an algorithm that solves X by a polynomial number of calls to $\mathcal{O}_X$.

```
1: procedure FIND A 3-COLORING OF  $G \in O(n^2)$ 
2:   3-Colorable Vertices  $S = \{\}$ 
3:   Ask the Oracle: "Is  $G'$  3-Colorable?"
4:   if The oracle says "NO" then
5:     End Algorithm. Graph is not 3-colorable.
6:   end if
7:   for Each pair of nodes  $u$  and  $v$  such that  $(u, v) \notin E(G')$  do
8:     Add edge  $(u, v)$  to  $E(G)$ 
9:     Ask the Oracle: "Is  $G'$  3-Colorable?"
```



```
10:   if The oracle says "NO", then
11:     else
12:       Contract  $u$  and  $v$  into one node
```

Self-Reduction Theorem

Every NP-Complete problem is self-reducible [5].

Cook Reductions

Let X and Y be two decision problems with input instance I_X and output instance I_Y , respectively. Given an oracle $\mathcal{O}_Y$ for problem Y , a **Cook Reduction** from problem X to problem Y is a polynomial-time algorithm algorithm for X that make can make use of use of the following subroutine many times:

- Let $I_Y(z) = \mathcal{R}_p(I_X(z))$
- Ask $\mathcal{O}_Y$ whether $I_Y(z)$ is "YES" or "NO"

Conclusions

- If any NP-Complete problem X is Karp reducible to problem Y then Y is NP-hard. If Y is also in NP , then Y is NP-complete.
- If, for any NP-complete problem X , if there exists a $Y \in NP$ such that $X \leq_p Y$, then $P = NP$.
- If $X \leq_p Y$ and we know of an algorithm, $\mathcal{A}_Y$ that solves Y , we can use a Cook reduction and $\mathcal{A}_Y$ to solve X .

References

- S. Har-Peled. Chapter 20: Polynomial Time Reductions. CS 473: Fundamental Algorithms Lecture Notes, UIUC, 2011. https://courses.grainger.illinois.edu/cs473/fa2011/lec/20_notes.pdf.
- S. A. Cook.
- M. R. Garey and D. S. Johnson. Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman and Company, San Francisco, 1979. The complexity of theorem-proving procedures. In Logic, Automata, and Computational Complexity: The Works of Stephen A. Cook, pages 143–152. 2023.
- S. Szeider. NP-Completeness Cartoon. TU Wien, Algorithms and Complexity Group, 2018.
- J. Erickson. Lecture 29: Self-Reducibility. CS 473: Algorithms Lecture Notes, UIUC, 2015.